In [1]:
```
import numpy as np
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score
```

In [2]:
```
digit_0 =[
    [1,1,1],
    [1,0,1],
    [1,0,1],
    [1,0,1],
    [1,1,1] ]
digit_1 =[
    [0,1,0],
    [1,1,0],
    [0,1,0],
    [0,1,0],
    [1,1,1] ]

digit_2 =[
    [1,1,1],
    [0,0,1],
    [1,1,1],
    [1,0,0],
    [1,1,1] ]

digit_3 =[
    [1,1,1],
    [0,0,1],
    [1,1,1],
    [0,0,1],
    [1,1,1] ]

digit_4 =[
    [1,1,1],
    [1,0,1],
    [1,0,1],
    [1,0,1],
    [1,1,1] ]

digit_5 =[
    [1,1,1],
    [1,0,0],
    [1,1,1],
    [0,0,1],
    [1,1,1] ]

digit_6 =[
    [1,1,1],
    [1,0,0],
    [1,1,1],
    [1,0,1],
    [1,1,1] ]

digit_7 =[
```

```
    [1,1,1],
    [0,0,1],
    [0,0,1],
    [0,0,1],
    [0,0,1] ]

digit_8 =[
    [1,1,1],
    [1,0,1],
    [1,1,1],
    [1,0,1],
    [1,1,1] ]

digit_9 =[
    [1,1,1],
    [1,0,1],
    [1,1,1],
    [0,0,1],
    [0,0,1] ]
```

In [3]:
```
x=np.array([digit_0,digit_1,digit_2,digit_3,digit_4,digit_5,digit_6,digit
```

In [4]:
```
x=x.reshape(10,15)
```

In [5]:
```
print("Training Data Shape :- ",x.shape)
```

```
Training Data Shape :-  (10, 15)
```

In [6]:
```
y=np.array([0,1,2,3,4,5,6,7,8,9])
```

In [7]:
```
model=MLPClassifier(hidden_layer_sizes=(100,),
    activation='relu',learning_rate_init=0.001,max_iter=1000)
```

In [8]:
```
model.fit(x,y)
```

Out[8]:

MLPClassifier

Parameters

hidden_layer_sizes	(100,)
activation	'relu'
solver	'adam'
alpha	0.0001
batch_size	'auto'
learning_rate	'constant'
learning_rate_init	0.001
power_t	0.5
max_iter	1000
shuffle	True
random_state	None
tol	0.0001
verbose	False
warm_start	False
momentum	0.9
nesterovs_momentum	True
early_stopping	False
validation_fraction	0.1
beta_1	0.9
beta_2	0.999
epsilon	1e-08
n_iter_no_change	10
max_fun	15000

In [9]:
```
pred=model.predict(x)
```

In [10…
```
pred
```

Out[10]:
```
array([0, 1, 2, 3, 0, 5, 6, 7, 8, 9])
```

In [11…
```
accuracy=accuracy_score(pred,y)
print("Accuracy :- ",accuracy*100)
```

```
Accuracy :-  90.0
```

In [12…
```
newp=[
    [1,1,1],
    [1,0,0],
    [1,1,1],
    [1,0,1],
    [1,1,1] ]
```

In [13…
```
z=np.array([newp])
```

In [ ]:
```

```